

Phase 4 — 任务队列与调度基础设施

SPEC-009 | Status: 设计中 | 依赖: SPEC-003 (Redis/MongoDB) , SPEC-004 (Agent Service 生成任务)

目标

实现 Agent 系统的异步执行基础设施: Task Queue (Redis Stream)、Worker Pool、任务生命周期管理 (取消/进度/通知)、Cron 调度器。这些是通用异步编排层——不与 Agent 业务逻辑耦合。

前置依赖检查

前置 Spec	状态	备注
SPEC-002	✅/❌	CI Pipeline 就绪
SPEC-003	✅/❌	Redis (Stream + Pub/Sub)、MongoDB (agent_tasks 集合) 可用
SPEC-004	✅/❌	Agent Service 生成 AgentTask 并入队

背景

Roadmap Phase 2 Week 4 (P2-12 ~ P2-16) 建立异步任务基础设施, Phase 4 Week 7 (P4-01 ~ P4-04) 扩展定时调度能力。SPEC-004 定义 AgentTask 数据模型和入队接口, 本 spec 实现队列消费、Worker 调度、任务生命周期管理。

详细设计

1. Task Queue (Redis Stream)

- 使用 Redis Stream, Consumer Group 模式保证 at-least-once 消费
- Stream Key: `agent:task:queue`
- Consumer Group: `worker-pool`

- 消息体 (JSON):

```
{
  "task_id": "uuid",
  "session_id": "uuid",
  "user_id": "uuid",
  "type": "agent_exec | scheduled_exec",
  "skill_chain": ["sql_executor", "stats_engine", "knowledge_search"],
  "params": {},
  "created_at": "ISO8601"
}
```

- Pending 消息处理: Worker 崩溃重启 → 从 Last Delivered ID 继续消费
- 死信队列: 重试 3 次仍失败 → 移入 `agent:task:dlq`

2. Worker Pool

- Goroutine Pool: 可配置并发数 (默认 4), 通过 channel 控制
- Worker 生命周期:
- 从 Redis Stream `XREADGROUP` 拉取消息
- 解析 AgentTask → 调用 Agent Service 内部的 `Execute()` 方法
- 执行结果写入 MongoDB `agent_tasks` 集合
- `XACK` 确认消费
- 优雅关闭: 收到 SIGTERM → 等待当前任务完成 (最多 30s) → 退出
- 健康检查: Worker 定期 heartbeat → Redis `worker:{id}:heartbeat`, TTL 15s

3. 任务生命周期管理

状态机

```
pending → queued → running → completed
      ↘ failed → retrying → failed (max retries)
```

取消机制

- Context Cancel: 任务执行时挂载 `context.WithCancel`
- 外部取消: `PUT /tasks/{task_id}/cancel` → Redis pub/sub 广播 cancel 信号
- Worker 收到 cancel → `ctx.Done()` → 清理资源 → 状态设为 `cancelled`

进度上报

- WebSocket 通道：客户端连接 → 订阅 `task:{task_id}:progress`
- Worker 定时汇报：`{step: 3/5, message: "正在执行统计分析..."}`
- 前端实时渲染进度条

完成通知

- Email 通知：任务完成后发送到用户注册邮箱（gmail）
- In-app 通知：WebSocket 推送 `task_completed` 事件
- 通知模板：`{task_name} 已完成, 耗时 {duration}, 查看结果`

4. 任务持久化

- MongoDB `agent_tasks` 集合：

```
{
  "_id": "ObjectId",
  "task_id": "uuid",
  "session_id": "uuid",
  "user_id": "uuid",
  "type": "agent_exec",
  "status": "completed",
  "skill_chain": [],
  "result": {},
  "error": null,
  "progress": {},
  "created_at": "ISODate",
  "updated_at": "ISODate",
  "completed_at": "ISODate",
  "duration_ms": 12345
}
```

- TTL 索引：`created_at` 字段设置 `expireAfterSeconds: 2592000`（30 天自动清理）

5. Scheduler 调度器（Phase 4 实现）

- robfig/cron v3：标准 cron 表达式解析 + 秒级精度
- 调度流程：
- 用户通过 API 创建 ScheduledTask → 持久化 MongoDB `scheduled_tasks`
- Scheduler 加载活跃调度任务 → 注册到 cron engine

- Cron 触发 → 创建 AgentTask → 入队 Redis Stream (复用 Task Queue)
- 暂停/恢复/删除: `PUT /scheduled-tasks/{id}/pause` 等

ScheduledTask 数据模型

```
{
  "scheduled_task_id": "uuid",
  "user_id": "uuid",
  "name": "每日销售分析",
  "cron_expr": "0 9 * * 1-5",
  "skill_chain": ["sql_executor", "stats_engine"],
  "params": {},
  "status": "active | paused | deleted",
  "last_run_at": "ISODate",
  "next_run_at": "ISODate",
  "created_at": "ISODate"
}
```

失败重试与超时

- 重试策略: 失败 → 延迟 1min / 5min / 15min 重试, 最多 3 次
- 超时终止: 单次调度任务最长执行 30min → Context Deadline 强制终止
- 连续失败 3 次 → 状态设为 `paused` + 通知用户

可行性分析

检查项	结论
是否需要新 DB 集合	Yes (agent_tasks, scheduled_tasks)
是否影响现有 API	Yes (新增 <code>/tasks</code> , <code>/scheduled-tasks</code> 路由组)
性能影响	Worker Pool 并发数需根据机器核数调整
是否需要新增 Skill	No (基础设施层, 非 Skill)
是否需要 E2E 测试	创建任务→进度推送→完成通知 UI 用例

相关文件

文件	角色	变更幅度
<code>internal/queue/stream.go</code>	Redis Stream 队列封装	新建
<code>internal/queue/dlq.go</code>	死信队列	新建
<code>internal/worker/pool.go</code>	Worker Pool 管理	新建
<code>internal/worker/runner.go</code>	Worker 任务执行器	新建
<code>internal/worker/heartbeat.go</code>	Worker 健康检查	新建
<code>internal/scheduler/cron.go</code>	Cron 调度器 (Phase 4)	新建
<code>internal/scheduler/manager.go</code>	调度任务管理 (Phase 4)	新建
<code>internal/domain/task/</code>	Task 领域模型	新建
<code>internal/service/task/</code>	Task HTTP API	新建

验证标准

1. 创建 AgentTask → Redis Stream 可见消息 → Worker 消费执行 → 结果持久化
2. Worker 崩溃 → 重启后从 Pending 消息恢复 → 幂等消费
3. 任务执行中 `PUT /cancel` → Context Cancel → 状态 `cancelled`
4. WebSocket 连接 → 订阅 `task:{id}:progress` → 收到进度推送
5. 任务完成 → Email + In-app 通知到达
6. Cron 表达式 `0 9 * * 1-5` → 每个工作日 9:00 触发 → 创建 AgentTask
7. 连续 3 次调度失败 → 状态自动 `paused`
8. Worker 停止 heartbeat → 15s 后被健康检查感知